

COMPUTER VISION

Report: Image Processing & Analysis Toolkit (GUI in Python + OpenCV + Streamlit)

1. Introduction

1.1 Background

Image processing is one of the most essential fields of computer vision and artificial intelligence. It deals with the manipulation and transformation of digital images to extract useful information, enhance quality, or prepare them for further analysis. With the availability of libraries like **OpenCV** and GUI frameworks like **Streamlit**, it is now possible to build interactive applications that allow users to experiment with real-time image transformations.

1.2 Objective of the Project

The main goal of this project is to design and implement a **GUI-based Image Processing Toolkit** where users can:

- Upload an image and visualize it.
- Apply transformations such as rotation, scaling, and translation.
- Perform filtering, morphological operations, and edge detection.
- Convert between different color spaces.
- Save processed images in multiple formats with compression.

1.3 Tools and Technologies

- **Python**: Core programming language.
 - **OpenCV**: For image processing operations.
 - **NumPy**: For numerical operations and array handling.
 - **Streamlit**: To design the GUI interface.
 - **Matplotlib**: For plotting histograms and visualizations
-

2. Fundamentals of Image Processing

2.1 Digital Image Representation

A digital image is a two-dimensional signal represented as a **matrix of pixels**. Each pixel carries intensity values which can be:

- **Grayscale**: Single channel, values between 0–255.

- **Color (RGB):** Three channels – Red, Green, and Blue.

2.2 Sampling & Quantization

- **Sampling:** Selecting the number of pixels in spatial dimensions (resolution).
- **Quantization:** Mapping continuous intensity values to discrete levels (bit depth).

Example:

- A 1-bit image → 2 levels (black & white).
- An 8-bit image → 256 levels (0–255).

2.3 CMOS vs CCD Sensors

- **CCD (Charge Coupled Device):** High-quality, low noise, but costly and power-hungry.
 - **CMOS (Complementary Metal Oxide Semiconductor):** Cheaper, lower power, integrated with processors, widely used in modern smartphones.
-

3. System Design & Architecture

The project is designed in modular form with the following architecture:

1. **Input Module:** Upload image (any format – JPG, PNG, BMP).
2. **Processing Core:** Image processing functions (filters, transformations, edge detection, etc.).
3. **GUI Layer (Streamlit):** Provides interactive elements such as sliders, checkboxes, and dropdowns.
4. **Output Module:** Displays processed image and allows saving in different formats.

3.1 GUI Layout

- **Left Sidebar:** Selection of operations.
- **Main Area:** Side-by-side display of original vs processed image.
- **Status Bar:** File size, resolution, channels, and format.

This modular design ensures separation of concerns and makes it easy to add new functionalities in the future.

4. Color Conversions & Transformations

4.1 Color Space Conversions

Color plays an important role in image analysis. The toolkit supports:

- **RGB ↔ Grayscale:** Removes color, retains intensity.
- **RGB ↔ HSV (Hue, Saturation, Value):** Separates color from brightness.
- **RGB ↔ YCbCr:** Used in compression (JPEG format).
- **RGB ↔ BGR:** OpenCV default storage order.

4.2 Geometric Transformations

- **Scaling:** Resize image (up-sampling or down-sampling).
- **Rotation:** Rotate around a pivot.
- **Translation:** Shift image in X and Y direction.
- **Affine Transform:** Preserves parallel lines (rotation + scaling + shearing).
- **Perspective Transform:** Simulates depth by mapping quadrilaterals.

These transformations are implemented using **cv2.warpAffine** and **cv2.warpPerspective** in OpenCV.

5. Filtering, Morphology & Enhancement

5.1 Filtering

- **Mean Filter:** Smooths image by averaging neighbors.
- **Gaussian Filter:** Reduces noise with Gaussian kernel.
- **Median Filter:** Removes salt-and-pepper noise.

5.2 Morphological Operations

Applied on binary images:

- **Erosion:** Shrinks white regions.
- **Dilation:** Expands white regions.
- **Opening:** Erosion followed by dilation (removes noise).
- **Closing:** Dilation followed by erosion (fills gaps).

5.3 Image Enhancement

- **Histogram Equalization:** Improves contrast.
- **Contrast Stretching:** Expands dynamic range.
- **Sharpening:** Highlights edges using Laplacian filters

6. Edge Detection & Compression

6.1 Edge Detection Techniques

- **Sobel Filter:** Detects gradients in horizontal/vertical direction.
- **Laplacian Filter:** Captures second-order derivatives.
- **Canny Edge Detector:** Multi-stage algorithm for accurate edge detection.

6.2 Image Compression

- Save processed images in different formats:
 - **JPG (Lossy compression, smaller size).**
 - **PNG (Lossless, larger size).**
 - **BMP (Uncompressed, very large).**

The toolkit compares file sizes in the status bar to illustrate trade-offs between quality and size.

7.Results, Applications & Conclusion

7.1 Results

The developed toolkit successfully allows:

- Upload and display of images.
- Application of transformations, filters, and enhancements.
- Comparison of original vs processed images.
- Export in different formats with compression analysis.

7.2 Applications

- **Medical Imaging:** Noise removal, edge enhancement.
- **Surveillance:** Motion detection, object recognition.
- **Robotics:** Real-time vision processing.
- **Media Industry:** Image editing and enhancement.

7.3 Conclusion

This project demonstrates the **power of Python + OpenCV + Streamlit** in building interactive and educational applications for computer vision. The modular design ensures that additional features (like video processing, AI-based segmentation, or face detection) can be integrated easily.